

Climate Change Knowledge Portal

Recommendations

For the Developer

Enerqa Consultancy

07/05/2025



Document Ref No.: CC1151TP525TCFIN

Prepared for: Earthna Centre for Sustainable Future

This document presents our comments on the portal, and what the developer of the portal should consider.

Copyright © Enerqa Consultancy 2025. All rights reserved.

This document contains confidential information that is the property of Enerqa Consultancy. No part of this document may be reproduced, distributed, or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording, or other information storage and retrieval systems, without the prior written permission of Enerqa Consultancy.

Table of Contents

1. Sitemap Component Descriptions	1
1.1. Top-Level Section	1
1.1.1. About	1
1.1.2. Qatar's Climate Actions.....	2
1.1.3. Events and News.....	3
1.1.4. Country Profiles	Error! Bookmark not defined.
1.1.5. Climate Data	7
1.1.6. Policy and Reports	Error! Bookmark not defined.
1.1.7. Community.....	8
1.1.8. Resources and Tools	Error! Bookmark not defined.
2. Data Management and Integration Requirements	9
2.1. External API Integration (UNFCCC, CCKP, ICAT, WRI)	9
2.2. Ingestion of National/Regional Data.....	10
2.3. Metadata/Tagging Taxonomy for Search and Indexing.....	11
2.4. Infographic/Media Handling Capabilities	12
2.5. User Content Submission Workflow with Moderation.....	13
2.6. AI-Powered Indexing and Smart Search	13
2.7. English–Arabic Multilingual Support.....	14
3. Technical Architecture Recommendations.....	15
3.1. Scalable CMS & Knowledge Base Backend.....	15
3.2. Tableau Dashboard Embedding.....	15
3.3. GPT-Style Search Integration	16
3.4. API/Webhook Integration with External Platforms	17
3.5. Responsive, Modular Frontend (Scalable Across MENA)	17
3.6. Compliance with Open Data Standards (CSV, JSON, XML, GeoJSON)	18
3.7. Real-Time Data Display for Climate Indicators	19

Climate Change Knowledge Portal

Technical Documentation & Recommendations

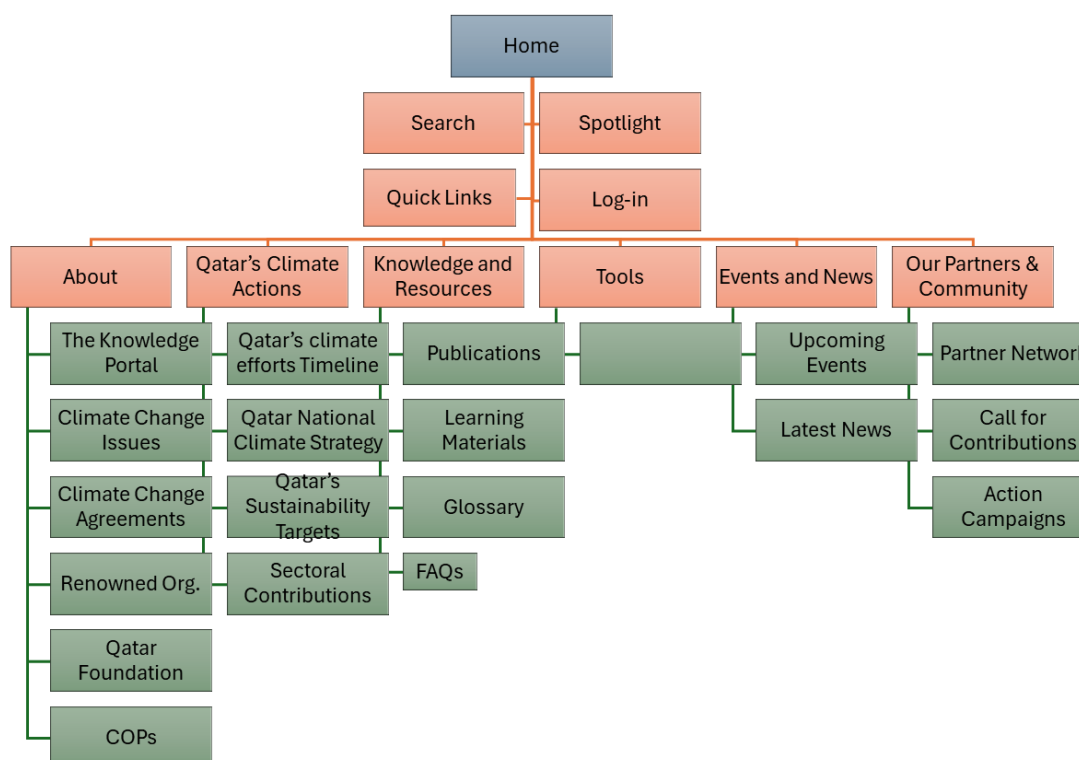


Figure: Sitemap structure for the Climate Change Knowledge Portal, with top-level navigation categories (orange) such as About, Climate Data, etc., and nested sub-pages (green) under each. This hierarchical design provides intuitive navigation and grouping of related content. In the sections below, each component is described along with its functionality and data characteristics.

1. Sitemap Component Descriptions

1.1. Top-Level Section

1.1.1. About

1.1.1.1. The Knowledge Portal, Earthna and QF

An introductory page that explains the portal's purpose, scope, and how to navigate it. This is primarily static content maintained by content managers, updated occasionally when portal features change. It may include overview infographics or a brief tutorial video for user onboarding (media support, but minimal interactive elements). No external data sources are required for this page.

Recommended metadata tags include descriptors like "About the Portal", "Overview", and related keywords, to facilitate semantic search discovery of this introductory content.

An informational page about Qatar Foundation and Qatar Foundation's involvement in climate initiatives. It details QF's climate research, funding programs, or relevant projects. This is static content, and updated when QF launches new climate programs (occasional updates). It may include images of QF projects or quotes from QF leadership (media content, static). No external data integration is needed here; it's an internal profile page. Use metadata tags such as "Qatar Foundation", "stakeholder", and "About Qatar Foundation" so that a query for "QF climate efforts" will retrieve this page. This page can also be tagged under an "About" category and cross-linked to relevant content (like if QF published a climate report, it should link to Publications).

1.1.1.2. Climate Change Issues

A section outlining key climate change topics relevant to Qatar and the global context (e.g. extreme heat, sea-level rise, desertification). It serves as an educational knowledge base page with richly formatted static content (text, images, possibly infographics) explaining each issue. Content managers will update it as scientific consensus evolves, but it's not as frequently updated as data sections (semi-static). The page might embed illustrative charts or images (e.g. an infographic on climate impacts) but generally does not require live data feeds. No direct API integration is needed, though it could link to external authoritative sources for further reading. Metadata should include thematic tags (e.g. "heatwaves", "drought", "climate impacts") so that an AI-powered search can retrieve this page when users query those topics.

1.1.1.3. Climate Change Agreements

A page detailing major global climate agreements (e.g. the Paris Agreement, Kyoto Protocol, UNFCCC framework). It provides summaries of each agreement and Qatar's commitments under them. The content is largely static text, possibly with document downloads or links to official agreement texts. It will be updated only when new agreements emerge or existing ones are significantly amended (infrequent updates). User interaction is limited to reading and clicking reference links; however, embedding PDFs or summary infographics of the agreements is supported (media content). No external API is needed since the agreements are historical documents, though links to UNFCCC or treaty databases can be provided. Use metadata like agreement names, years, and keywords (e.g. "Paris Agreement 2015", "global treaty") for indexing, ensuring that a semantic search for "climate treaties" or a specific agreement will surface this page.

1.1.1.4. Renowned Organizations

A directory of major organizations and institutions active in climate change (e.g. IPCC, UNEP, WRI, regional NGOs). This page lists organizations with a brief description of their role and a link to their websites. It's mostly static content but should be **dynamic** in the sense that new organizations can be added over time by content managers. The page might include logos or insignia for each organization (media elements), possibly presented in a grid or list. No external API is required; content is curated manually, though an RSS feed or API could theoretically pull organization news (not in scope here). Metadata should include organization names and categories (e.g. "research", "policy", "finance") to assist search. Tagging organizations in a controlled vocabulary will also help the AI search tool relate content (for example, if a user searches "UNFCCC initiatives", the system can link to this section because UNFCCC is tagged here).

1.1.2. Qatar's Climate Actions

1.1.2.1. Qatar's Climate Efforts Timeline

An interactive timeline of Qatar's key climate actions and milestones. It chronologically showcases events such as policy enactments, project launches, international pledges, etc. This timeline is **dynamic**, regularly updated as new milestones occur (e.g. a new climate law in 2025 would be added). Developers should implement this as either an interactive UI component (allowing users to scroll or slide through years and events) or a well-organized chronological list. Users might interact by clicking an event to see details or media (e.g. a photo of a conference or an embedded video clip if available). The content for each event is managed via the CMS (title, date, description, media attachments). No external API is strictly required since events are curated, but each event entry might link to external references (like a news article or official announcement). For search, each timeline entry should be indexed by its title, date, and keywords (e.g. "National Climate Strategy launch 2021") so that the AI search tool can retrieve specific events. Tag events with categories such as "Policy", "Initiative", or "International Engagement" to enable filtered search results (for example, a search for Qatar's policy history could filter all timeline events tagged as policy).

1.1.2.2. Qatar National Climate Strategy

A dedicated page outlining Qatar's National Climate Change Strategy (NCCS). It likely summarizes the strategy's pillars, goals, and priority actions. This page is mostly static textual content (the strategy document is a fixed reference), but may be updated if the strategy is revised or if progress updates are included over time. The page could include downloadable PDFs of the strategy or an executive summary infographic. It might also break down the strategy into sub-sections (mitigation, adaptation, etc.) for readability. No live external data is required, but it could dynamically pull in *related content* from elsewhere in the portal (for instance, showing links to relevant "Policies and Reports" or "Sustainability Targets" that align with the strategy). To integrate with other systems, one might link this to UNFCCC's registry if Qatar submitted this strategy officially, but that's just a hyperlink rather than an API. Important metadata includes tagging this page as "National Policy", "Climate Strategy", and linking it to keywords like "NCCS" and related targets or sectors mentioned. This allows the semantic search to surface the strategy page when queries like "Qatar climate plan" or "national climate policy Qatar" are made. Additionally, aligning metadata with UNFCCC terminology (e.g. tagging it as an "NDC" if applicable or "National Strategy") can help future integration or cross-referencing.

1.1.2.3. Qatar's Sustainability Targets

A page listing Qatar's specific climate and sustainability targets – for example, greenhouse gas emission reduction percentages by 2030, renewable energy capacity goals, water conservation targets, etc. This is a dynamic content page in that the *values* of targets might be static (set by policy) but the **progress** toward those targets can be updated. For each target, developers can present the target value and possibly a visual indicator of progress (e.g. a progress bar or gauge showing current status vs target). To support this, the page might pull data from relevant sources: for instance, if a target is "X% renewable energy by 2030", the current percentage could be fetched from an internal database or an API (maybe an annual report or data source providing the current stat). Integration with external APIs could occur here – e.g. pulling the latest national emission levels from UNFCCC or a WRI dataset to show progress against an emission reduction target. If live integration is complex, an alternative is to allow content managers to input the latest stats periodically (semi-automated updates). The page should definitely support interactive charts or at least up-to-date infographics for each target (user interaction in terms of viewing details for each target's progress). Metadata for each target should include the

theme (emissions, energy, etc.), the numerical target, target year, and status. Tagging these allows the AI search to answer questions like “What are Qatar’s climate targets?” or “renewable energy goal in Qatar” by pointing here. It’s also useful to use consistent units and terminology (for example, tag “emissions (MtCO₂)” so the search engine can relate a question about CO₂ to this content).

1.1.2.4. Sectoral Contributions

This page details how different sectors contribute to Qatar’s greenhouse gas emissions and climate efforts. It breaks down data by sector (Energy, Transportation, Industry, Waste, etc.), showing each sector’s emission share or specific initiatives in that sector. This is a data-rich section – expect charts like pie charts of GHG emissions by sector, or bar charts of mitigation actions by sector. The content can mix static explanatory text (describing each sector’s role and policies) with **dynamic data visualizations**. For example, using data from Qatar’s GHG inventory (which could be integrated via the UNFCCC data or WRI’s Climate Watch API) to populate an emission chart that updates when new inventory data is released. If integration is possible, set up an API call or data ingestion process for updated sectoral emission numbers annually. If not, content managers can update the values yearly. Users should be able to interact by toggling between different metrics (e.g. emissions vs. mitigation investments by sector) or viewing trends per sector. External API integration: likely yes, to fetch structured data on emissions or sector metrics (e.g., the UNFCCC API provides GHG inventory data, or WRI Climate Watch provides sectoral emissions datasets). Even if data is ingested and stored locally, aligning with those external sources ensures accuracy. Metadata: each sector should be a tag (e.g. “Energy”, “Transport”), and data years and units should be documented. This way, a search query like “Qatar emissions from transport sector” can directly retrieve or be answered by this page, and the AI search tool can link queries about sectors to this content. Also, cross-tagging with related content (for instance, if there’s a National Policy about the energy sector, both that policy and this sectoral page should share the “Energy” tag) will enhance the semantic network of the site.

1.1.3. Events and News

1.1.3.1. Upcoming Events

A calendar or listing of climate-related events, with a focus on events in Qatar or the MENA region. This might include conferences, workshops, webinars, community activities, or international events like upcoming COPs that are relevant to the audience. The content is **highly dynamic** – events will be continuously added and expire after their date passes. The portal should allow content managers (or possibly authorized community members) to input new events with details such as title, date & time, location (physical or virtual), a brief description, organizer, and a link for more information or registration. A good implementation would be to display events in chronological order (soonest first) and possibly a calendar view or filters by month. For past events, we might either drop them from the list or archive them under a past events section if needed (the sitemap doesn’t specify an archive, so probably just remove past ones). An important feature is to ensure events sort automatically by date, which means the backend should store event dates in a date field and the frontend can filter out old dates. We could integrate an **iCal feed or Google Calendar API** if we want users to subscribe or if the organizers already maintain a calendar, but at minimum a manual system is fine. If the portal wants to auto-ingest events from partner organizations, a webhook or RSS mechanism could be set up (for example, if QF or MoECC publishes events on another site, we can fetch those). For now, assume manual entry for simplicity but design with an API in mind for future scaling. User interaction: users can click an event to get more details. Perhaps allow adding to personal calendar (download .ics). If login is supported on the portal, users might mark interest in events or sign up, but that’s extra functionality (not explicitly asked, except maybe in community engagement context). Metadata: tag events by type (“Conference”, “Training”, “Youth Event”), by location (especially if we might later filter by national vs international), and by thematic focus if applicable (“energy”, “adaptation” if the event has a theme). This tagging helps if a user searches “training events on climate adaptation” – the search could match those tags. The **Upcoming Events** page itself should be easy to find and possibly highlighted on the homepage (often events are something one might highlight). Technically, implementing this might involve a content type “Event” in the CMS and a custom frontend component to nicely display event info and maybe map integration (if physical events, a map thumbnail showing location could be nice). The design must also accommodate bilingual content: events in Arabic should be enterable or translatable (title/description in Arabic, especially if targeted at local community). Alternatively, if an event is inherently bilingual (like an international conference), we might just have one entry and show it regardless of language, but ideally provide translation for the description.

1.1.3.2. Latest News

A news section aggregating climate change news and updates relevant to Qatar and the region. This will function like a blog or news feed. Content managers (or possibly an automated feed) will post news articles frequently – these could be original news written by the portal team (e.g., covering local climate initiatives, summarizing international news with local perspective) or curated from external news sources (like re-posting a relevant article snippet with credit). Given that the portal is likely run by MoECC and QF, it may lean towards **original or officially endorsed news** rather than an automatic aggregator, to ensure accuracy and relevance. Thus, we might implement it as a CMS news post type (title, date, body, source link if any, images). Frequency could be weekly or as news breaks. This is clearly dynamic content. For user engagement, each news item should allow rich media: expect images for news articles (like photos from an event or charts from a report), possibly videos. The front-end should show recent headlines maybe with thumbnail images, and allow clicking through to full stories. Pagination or infinite scroll would be needed if a lot of news

accumulates. We should also allow users to filter or search within news (by date or topic tags) because over years this could become a large archive. A potential integration is to use an RSS feed from known sources (like “Science & Environment” news from local media) and either auto-post those or use them as references for content managers. However, caution with automation: it might pull in unrelated content, so editorial oversight is better. The site’s **Search** (both keyword and AI search) should definitely index the news articles. That means each news post’s text will be in the search index. Metadata for news: tags for topics (e.g., “Climate Finance”, “Solar Energy”, “Air Quality”) and perhaps for geography (if it’s international vs Qatar news). Also include date as a search facet if we implement advanced search (e.g., user can say “news 2024”). The AI search tool might answer a question like “What happened in the last COP?” by pulling from a news article covering that COP, so it’s crucial the news content is well-structured and labelled for retrieval. From a technical perspective, a robust **CMS or blogging platform** capability is needed here: authors, timestamps, and possibly the ability to schedule posts. If user login exists, maybe allow comments on news for community interaction – but that introduces moderation overhead (not explicitly requested, so can be omitted initially). Bilingual support: likely we will produce news in both English and Arabic to serve all audiences. That means possibly separate posts for each language, or using a CMS that can have bilingual fields and then listing accordingly. The interface should allow switching language and seeing the corresponding set of news (or a combined feed if desired, but probably separate feeds for clarity). To keep local users engaged, ensure the Arabic news is just as frequently updated as English. All in all, **Latest News** keeps the portal fresh and current, encouraging repeat visits and providing an authoritative climate news source for Qatar.

1.1.3.3. COPs

A page dedicated to the Conference of the Parties (COP) climate summits, highlighting key outcomes and Qatar’s participation. It could list recent COPs (e.g. COP26, COP27, COP28) with brief summaries and any commitments or statements by Qatar. This content is **dynamic** on an annual cycle – after each COP, content managers will add the latest conference outcomes. The page might use a timeline or accordion UI for each year’s COP, and include photos or summary infographics from the events (media-heavy content). While primarily manual content, it may link to external resources like the UNFCCC COP reports; no direct API integration is expected for text, but an API could be used to pull in specific data (for example, the CO₂ pledges database, if needed for each COP – though likely not in this summary page). Metadata should include “COP”, the year or number (e.g. “COP28”), and topics like “climate conference”, so a search for “COP outcomes” or “international climate conference” finds this page. Each COP entry could also be tagged with relevant themes (e.g. “adaptation, finance”) if detailed, which improves semantic linking (e.g. searching “climate finance discussions” could match a COP summary that mentions finance).

1.1.4. Knowledge and Resources

1.1.4.1. Publications

A repository of reports, research papers, articles, and publications related to climate change, especially those relevant to Qatar or produced by Qatar’s institutions and partners. This can include official government reports (e.g., National Communications to the UNFCCC), academic research papers (from QF or local universities), policy briefs, and publications from international bodies that have local relevance. The content here is document-centric. Each entry likely has a title, authors, publication year, a brief abstract or description, and a link to download or view the full publication (PDF or external link if it’s on another site). We should allow sorting or filtering by year and topic. Over time, this library will grow, so having a robust indexing is key. This section is somewhat dynamic: new publications will be added when available, but not on a daily basis – maybe a few per quarter. The CMS should handle file uploads (for locally hosted PDFs) and store metadata for each publication. A possible enhancement is integration with a bibliographic database or Google Scholar for citation info, but that’s not necessary; a manual entry works. For search and semantic indexing, publications are rich sources of info, but often lengthy. The AI search might pull info from them if they’re indexed – to manage this, we might consider indexing the text of PDFs as well (OCR or extracting text if PDFs are text-based). If that’s feasible, it could immensely strengthen the AI tool’s ability to answer detailed questions (though it’s a heavier task). At minimum, the abstract and title should be indexed and tagged. Metadata tagging: classify publications by category (“Research”, “Policy Report”, “Case Study”) and by subject tags (“climate finance”, “biodiversity”, etc.). Also, tag with year and author organization if relevant (e.g., “Qatar University”). That way, queries like “research from 2022 Qatar climate” can filter appropriately. Also, link publications with other content – for instance, if a publication is about solar energy, and we have a **Tool** on solar, cross-reference them. Bilingual support: Most publications might be in English, but if there are Arabic publications or translations, include those. It might be wise to have language as a filter or separate listing (some official reports might have both English and Arabic versions). The metadata should note the language of the publication. If an Arabic user searches the Arabic term for something and there’s an English publication, the AI might still find it if we have translated metadata, but ideally we encourage content in both languages. **Publications** section will be a cornerstone for more in-depth information, so technically ensuring easy retrieval (via search or browsing) is a priority—this could mean implementing a search within publications or simply relying on the main search.

1.1.4.2. Learning Materials

A library of educational resources related to climate change. This can include tutorials, courses, presentations, infographics, videos, and toolkits that are intended for learning and capacity building. The portal might either host these directly or link to external sources.

For instance, there might be PDF guides on conducting greenhouse gas inventories, slide decks from training workshops, or links to e-learning courses (like from UN CC:Learn or similar platforms). The content is dynamic in that new materials will be added as they become available (especially if the portal is used to distribute content from workshops or partners like academia). Each learning material entry should have metadata like title, type (PDF, Video, etc.), a short description, the source or author, and possibly an estimated level (e.g., “Beginner, Intermediate”). We should also categorize them (topic-wise: e.g., “Climate Science Basics”, “Adaptation Training”, “Data Analysis Tools”). Implementation: similar to a library or downloads section. We can have filters or categories listed so users can narrow down (for example, show only Videos, or show only materials about “policy making”). We might integrate with external platforms for content: e.g., embedding YouTube videos for recorded lectures, or linking to an external MOOC. If so, ensure the embed is responsive and accessible. If materials are files hosted on the portal, use a robust file management (possibly storing in a cloud storage or the CMS’s media library) and ensure links are not broken. This section might also benefit from tagging materials with appropriate audience tags (like “for educators”, “for students”) if relevant. For search integration, all titles and descriptions should be indexed; an AI query like “climate change course for beginners” should surface relevant items here (which relies on good descriptions and metadata). Possibly, integrate a bit with the AI search by letting it know this section contains pedagogical content – so if someone asks “Where can I learn about X?”, the AI might say “Check the Learning Materials section” and list some items. Bilingual: Materials might be language-specific. Likely, some resources will be in English, some in Arabic. We should allow filtering by language or clearly mark which language each item is in. If possible, when adding content, provide translated titles/descriptions in the other language as well, but if a material itself is only in one language, that’s fine; just don’t show it in the other language listing unless desired. (Alternatively, list everything but mark language, though Arabic audience might prefer to only see resources they can utilize.) Ensuring the **Learning Materials** repository is well-organized will encourage knowledge dissemination and position the portal as a learning hub.

1.1.4.3. Glossary

A comprehensive glossary of climate change and sustainability terms, provided to help users (especially those new to the subject) understand technical jargon and acronyms used throughout the portal. This page will list terms alphabetically (or allow search within it) with succinct definitions. For example, terms like “NDC”, “Mitigation”, “Renewable Energy Certificates”, etc., would be defined. The content is relatively static, but it will **expand** over time as new terms appear in discourse or content (content managers should add entries whenever they use a new term in an article that might not be common knowledge). It’s important that the glossary is easily accessible – possibly via a link in the header or contextually (like tooltip definitions when hovering over terms in content, which could be an extended feature using the glossary data). Implementation-wise, treat glossary entries as a structured content type (Term, Definition, possibly Category). We might also link sources or further reading for each term if needed (e.g., link to Wikipedia or a source article) but typically a brief internal definition suffices. User interaction: primarily clicking a letter or typing in a filter box to find terms. We might incorporate anchor links (A, B, C, etc., to jump to those sections). Also, as a nice feature, integrate the glossary with the AI search – for instance, if someone asks “What is [some term]?”, the AI search tool can respond with the glossary definition. The AI could either retrieve the glossary page snippet or we could program a simple lookup for known terms. Tagging/metadata: not much needed beyond labeling it as “Glossary” – each term might have a tag for its category (like “Science”, “Policy”, “Finance” to classify terms). Bilingual support: we need the glossary in both English and Arabic. This is crucial because technical terms need proper translation. Some terms might not have a direct Arabic equivalent and will be transliterated, so we should include those in the Arabic glossary with explanations. Possibly, the Arabic glossary will list Arabic terms (or Arabic transliterations) alphabetically in Arabic order. We might maintain the glossary as two separate lists, one per language, due to different scripts and ordering. The system should ensure that if the user switches language, they see the glossary relevant to that language. If a term is language-specific (like an English acronym “UNFCCC”), the Arabic glossary might list it under the Arabic alphabet corresponding to “U” sound or simply under Latin section – we need to handle that thoughtfully (maybe a unified bilingual glossary interface could be complex, so separate is fine). Ensuring the glossary is populated and updated will greatly enhance user experience and also reduce confusion that could arise in the AI search (since definitions are clearly provided, the AI can use them as canonical answers for “define X” type questions).

1.1.4.4. FAQs

A Frequently Asked Questions section addressing common inquiries about climate change and about the portal itself. This is a user-friendly way to provide quick answers to things many people ask. It likely includes questions like “What is climate change and how does it affect Qatar?”, “How do I use the data explorer?”, “Where does the portal’s data come from?”, or “How can I contribute to the portal?”. This content is mostly static text Q&A pairs, though it should be updated based on user feedback (dynamic in that new FAQs can be added as new common questions emerge). Implement it as an easy-to-navigate list, possibly grouped by categories (e.g., General Climate Questions, Portal Usage Questions, Data Questions). A nice UI feature is an accordion or collapsible list where clicking a question reveals the answer. Since the AI search tool exists, one might think FAQs are less necessary, but they serve as curated, concise answers and also feed the AI with correct responses. In fact, we can use the FAQ content to train or seed the semantic search. For example, ensure each Q and A is indexed; if a user question closely matches an FAQ question, the AI or search can respond with that answer directly. From a management perspective, content managers should monitor what users ask via the AI tool or contact forms, and update the FAQ section accordingly. Also, this section should be linked prominently (like if someone is lost

or needs help, FAQ helps them find answers without having to ask a human). If the portal has a helpdesk or support contact, mention it here as well (“Didn’t find your question? Contact us at...”). No external integration needed, it’s purely internal content. Metadata: not much beyond perhaps tagging each Q&A with a category label. The search should pick up both the question and answer text. For instance, if someone searches “data source update frequency”, an FAQ like “How often is the data updated?” should appear. Bilingual support: Provide the FAQs in both languages. Likely we’ll create corresponding Arabic Q&A pairs rather than auto-translating on the fly, because phrasing needs to be natural. Possibly, users might phrase questions differently in Arabic, so we might adjust the question wording to fit what an Arabic speaker might literally ask. This nuance can help the search match queries better. In summary, the **FAQ** section is a quick-reference knowledge base that complements the interactive AI search, ensuring users can get straightforward answers to common queries and learn to navigate the portal effectively.

1.1.4.5. Qatar (Country Profile)

A comprehensive profile page for Qatar’s climate data and indicators. This page is highly **dynamic** and data-driven, consolidating multiple types of information about Qatar. It typically includes sections such as climate trends (temperature, precipitation), greenhouse gas emissions, climate vulnerability indices, key climate policies, and perhaps socio-economic context relevant to climate. The profile should present interactive charts and maps – for example, a graph of Qatar’s GHG emissions over time, or a map of Qatar showing climate zones or impacts. To achieve this, the page will integrate with external data APIs: for historical climate data and projections, the World Bank’s Climate Change Knowledge Portal (CCKP) API can provide temperature/precipitation data and scenarios; for emissions and energy data, WRI’s Climate Watch or UNFCCC data can provide national GHG figures. These integrations might be done server-side (ingesting and caching data periodically) to ensure fast loading, with the page rendering the latest dataset. The **Country Profile** should also allow user interaction such as selecting different indicators (e.g. switch the emissions chart from total CO₂ to per capita, or view different time periods). The “Expandable” note in the sitemap suggests that in the navigation menu, **Qatar** can be expanded to reveal sub-categories or sub-pages (possibly detailed breakdowns like “Emissions”, “Impacts”, “Adaptation Measures”). If implemented, each sub-page would focus on a specific data aspect but remain under the Qatar profile umbrella. From a content management perspective, much of the content can be dynamically generated from databases or APIs, with content managers adding explanatory text or context for each section. This page will likely link with other parts of the portal: for example, it can pull **Latest News** related to Qatar or list **National Policies** (from the Policy section) relevant to Qatar’s profile. Metadata is critical: tag all data and content by country (here “Qatar”) and by indicator type (e.g. “GHG emissions”, “temperature trend”). Consistent country codes or IDs should be used (e.g., ISO country code “QAT”) in the backend to link data sources. With proper metadata, the AI search tool can answer queries like “What is Qatar’s current emission level?” by retrieving data from this profile. It can also link queries about Qatar’s climate (in Arabic or English) to this profile page. Multilingual support means all labels and text here need translation; data (numbers) remain the same, but units should be localized (e.g. decimal separators, etc., depending on locale).

1.1.4.6. Regional Profiles (Expandable)

Profiles for other countries or regions in the MENA (Middle East and North Africa) region, to extend the portal’s knowledge base beyond Qatar. “Regional Profiles” being expandable suggests a menu structure where users can select other geographic profiles. This could include individual country pages for other nations (e.g. Bahrain, Oman, etc.) or a group page like “GCC Region” or other regional aggregations. Initially, the project might focus on Qatar, but the architecture should accommodate adding more profiles easily. Each profile would be generated from data (similar to Qatar’s profile) but tailored to the selected country/region. To support this, developers should design a **template-driven** system: one profile page template that, given a country code or region identifier, pulls the relevant data and content. Data integration for these profiles will rely heavily on external open data APIs, since we may not manually curate all content for every country. For example, the World Bank CCKP provides climate data for all countries, and WRI Climate Watch provides emissions, NDC targets, etc., for many countries. By integrating these, the portal can automatically populate basic climate indicators for any country profile. Content managers can then add narrative or specific context for each country as needed (and translate it for bilingual support). Regional profiles (like “Gulf Cooperation Council (GCC) aggregate”) could compile data from multiple countries – potentially calculating sums or averages – which might require custom data handling or using sources like UNFCCC regional reports. User interaction: similar to Qatar’s page, users should be able to pick a country from a list or map, then view its data, possibly toggling indicators. No matter which country/region, the profile pages should maintain a consistent structure and design. Metadata: each profile must be tagged with its country or region name and code, and content within it tagged by topic. This ensures that a user searching for, say, “climate data Bahrain” will get to the Bahrain profile. Also, language-specific metadata (the country name in Arabic vs English) should be accounted for so that Arabic queries match the Arabic content. In summary, **Regional Profiles** extends the dynamic data-driven approach of the Qatar profile to a broader set, creating a scalable knowledge base across MENA.

1.1.5. Tools

This section serves as a hub for toolkits and interactive tools provided by the portal. It could encompass case studies of successful climate action, toolkits for implementation, and links to interactive tools that help users take action or make decisions. For example, this section might include content such as “Renewable Energy Best Practices in Qatar”, “Water Conservation Toolkit”, or “Carbon Footprint Calculator” as a tool. It may not have dedicated submenu pages listed in the sitemap (as of now), but internally it should organize content for ease of navigation. Some content here will be static articles (e.g., case studies or guidelines), possibly with media like images, diagrams, or embedded videos demonstrating solutions. Other content could be interactive – for instance, if there is a calculator or map-based tool, it could be embedded or linked here. The section should be **dynamic in content updates**: as new tools are developed (either by the portal team or partner organizations), content managers will add them. For instance, if a new AI tool is developed to predict solar energy output, this section would highlight that. Integration with external APIs might occur if we feature external tools (e.g., an embedded map tool might call an API for geospatial data, or an external carbon calculator might be wrapped in an iframe). We need to ensure that any such external tool integration is seamless (and perhaps provide fallbacks if the external service is down). For content structure, **Tools** could be presented as a series of cards or tiles on a landing page – each linking

to either a detail page or directly launching a tool. Since this is a broad category, a tagging system will help: e.g., tag each solution as “Mitigation” or “Adaptation” or “Technology”, etc., so users (and the AI search) can filter or distinguish them. A semantic search query like “energy solutions for climate” should ideally find the relevant items here. Additionally, include metadata about whether the item is a **Tool** (interactive) or a **Guide/Toolkit** (informational), which could be another facet for search filtering. In terms of user interaction, ensure that tools pages invite engagement – for example, a guide could have a PDF download of a handbook, or a tool might allow user input and produce some result (like the aforementioned calculator). From a development standpoint, building this section requires a flexible CMS structure to host both static content and embed dynamic widgets. It should be scalable for future additions, possibly including subsections if the content grows (e.g., later we might group solutions by sector or create a submenu once there are many). In summary, **Tools** is a largely content-driven section bridging knowledge and action – it consolidates resources that users (communities, educators, policymakers) can directly apply. Proper categorization and periodic updates will keep it relevant and useful.

1.1.6. Our Partners & Community

1.1.6.1. Partner Network

A section showcasing the network of partner organizations and stakeholders collaborating with the portal. This likely includes NGOs, academic institutions, government agencies, and possibly international partners in the climate domain. The page might display partner logos and names, and clicking on a partner could show a profile with more details (mission, projects, link to their website). It serves both as recognition of contributors and as a directory for users to discover organizations involved in climate action in the region. In terms of content, this is relatively static — partners don’t change daily — but it should be **maintainable** so that new partners can be added and information updated as partnerships evolve. We should implement a content type “Partner” with fields: name, logo image, description, category (e.g., Government, NGO, Research, Private Sector), and links (website, social media perhaps). On the front-end, all partners might be listed alphabetically or by category, or even plotted on a map if geographic. For user interaction, it’s mostly browsing; maybe allow filtering by category or search within partners if the list is large. External integration isn’t necessary here (we’re not pulling this via API; it’s internal data), but one idea: if partners have RSS feeds or news, we could in the future integrate partner news on their profiles, etc. For now, focus on making it easy to update. Moderation: If we allow, say, partners to submit their interest or info (not explicitly asked, but “Call for Contributions” might cover contributions from external people), the addition of a partner might be through a form then approved by admin. Ensure logos and media are handled correctly (we might want to auto-resize or have a CDN for images). Metadata: tag each partner entry with their focus area (“renewables”, “adaptation”, etc.) and type, and possibly location (country or city). That way, if someone searches “research organizations on climate in Qatar”, the AI could find partner profiles tagged “research” and “Qatar”. Also, link this section with relevant content: for example, if a partner contributed a **Case Study in Publications**, those pages should reference the partner and vice versa. This cross-linking can be facilitated by tagging or a small relational database linking partner IDs to content they provided. Bilingual support: each partner’s description should be translatable to Arabic, so the partner profiles should appear in the selected language. The **Partner Network** fosters a sense of community and credibility by showing the ecosystem of climate efforts that the portal is connected with.

1.1.6.2. Call for Contributions

This is an interactive page inviting the community (experts, researchers, students, organizations) to contribute content or data to the portal. It likely outlines the types of contributions sought – for example, submission of research articles, case studies, educational materials, data sets, or even success stories from community projects. The page should provide guidelines on how to contribute (content format, quality standards, any review process) and a mechanism to submit. Technically, that mechanism could be a **submission form** on the portal (where users fill in info and upload files or links). Alternatively, it could simply provide an email or contact for submissions, but an online form is more user-friendly and can tie into the backend workflow. We must implement a **user submission workflow with moderation** here: when someone submits content, it could create a draft content item in the CMS (flagged as “awaiting review”) that administrators can then review, edit, and approve to publish on relevant sections of the site (for instance, a submitted article might end up in **Publications** or a submitted case study in **Solutions and Tools**). This workflow may involve notifications (email the admins when a new contribution is submitted, and perhaps email the contributor when it’s published or if revisions are needed). For moderation, having a spam/inappropriate content check is important – maybe integrating a captcha or requiring login to submit could reduce junk submissions. On the **Call for Contributions** page, aside from the form, we should clearly state the licensing or usage policy of submissions (e.g., contributors agree their content can be edited and published). We should also highlight the benefit to contributors (credit, visibility, advancing climate knowledge, etc., to encourage participation). In terms of dynamic vs static: the page text (guidelines) is static, but the **submissions are dynamic** and feed into other sections. If contributions become frequent, we might even have a sub-page listing recent community contributions or a tag indicating “community-contributed” content. For search and metadata: contributions once published are like any other content and should be tagged accordingly by the editors. It might be useful to tag content with a flag like “community submission” internally. The AI search tool doesn’t need to treat them differently, but tracking them could be useful for administrators. Bilingual considerations: likely, contributions could be submitted in either English or Arabic. The portal team might choose to translate some contributions to the other language for broader reach. If that’s the case, the moderation workflow should include translation steps. At minimum, have

separate submission forms or language options so contributors provide content in the language they are comfortable with. Overall, **Call for Contributions** adds a participatory dimension to the portal, and the technical implementation must ensure user submissions flow smoothly into the content pipeline with proper oversight.

1.1.6.3. Action Campaigns

A section highlighting active campaigns or initiatives that aim to engage the public or specific groups in climate action. These could be things like a national tree planting campaign, a “green campus” challenge, a sustainability innovation contest, or community clean-up drives. The portal will use this page to promote such campaigns and encourage users to participate or follow along. Each campaign entry should have a description of the campaign, its objectives, how to participate, timeline, and maybe progress updates or results if ongoing. For example, an “Energy Conservation Challenge 2025” might show that it’s running from June to Dec 2025, how many people have joined, and tips on conserving energy. Technically, the content here is semi-dynamic: campaigns have a finite duration, and new campaigns will be launched over time. So content managers will add new campaigns and mark old ones as completed or remove them. If feasible, we could show active campaigns separately from archived past campaigns. User interaction could be more involved here: possibly integration of social media feeds or a hashtag to show campaign buzz, or a way for users to register interest (“Join Campaign” button that might link to an external sign-up or just display info). If we incorporate user accounts, campaign pages could allow users to log an action (for instance, “I planted a tree” submission which increments a counter). That’s an advanced feature and not strictly required, but something to consider in design (maybe in future phases). As for integration, if a campaign has its own external site or API (some global campaigns do, like a counter of actions), we could embed or fetch that data to display live progress. For example, if partnering with an app that tracks how many trees planted, an API could show the number live. In absence of that, content managers can update the progress manually (e.g., “50 schools participated as of August”). Metadata and tagging: each campaign should be tagged by theme (“afforestation”, “energy saving”, etc.) and by target audience if relevant (“youth”, “community”, “business”). This helps in search; a query “tree planting in Qatar” should hit the afforestation campaign. Also, tag it as “Campaign” to distinguish from static content. From a UI perspective, use engaging visuals – banners or photos from campaign activities – to draw interest. Bilingual content: campaign info must be in both languages to rally the widest audience. The structure can mirror between languages, but any user-submitted elements (if present, like comments or counts) need to aggregate regardless of language. **Action Campaigns** thus is a living part of the site that moves beyond knowledge to direct action, requiring an agile content approach and possibly interactive features to really succeed.

2. Data Management and Integration Requirements

2.1. External API Integration (UNFCCC, CCKP, ICAT, WRI)

To provide up-to-date and authoritative data, the portal will integrate with key external data sources via their APIs. This includes:

#	API / Data Source	Mapped Portal Section(s)	Purpose / Integration Function
1	UNFCCC	Climate Change Agreements, Global Agreements, Qatar’s Climate Efforts Timeline	Access to NDCs, Biennial Transparency Reports, COP decisions, and Article 6 registry for tracking global and national commitments.
2	WRI Climate Watch	Indicators and Statistics, National Policies, Country Profiles	Provides GHG emissions data, policy databases, NDC trackers, and energy transition insights for comprehensive country-level analysis.
3	World Bank Climate Change Knowledge Portal	Country Profiles, Regional Frameworks, Indicators and Statistics	Supplies climate data (e.g., temperature, precipitation projections) and vulnerability indices for risk assessments and regional planning.
4	ICAT / UNEP DTU	Policy and Reports, National Policies, Solutions and Tools	Offers MRV methodologies and implementation guidance for mitigation actions, supporting policy evaluation.
5	NASA POWER API	Indicators and Statistics, Climate Data	Delivers Qatar-specific time series data on temperature, solar radiation, wind, and precipitation for climate modeling.
6	Copernicus CDS / ECMWF	Climate Data, Indicators and Statistics	Provides climate model projections, anomalies, and datasets for building interactive dashboards.
7	NOAA CDO API	Climate Data, Indicators and Statistics, Global Agreements	Supplies historical climate datasets, heatwave stats, and rainfall events for trend analysis and agreement tracking.
8	OpenAQ	Indicators and Statistics, Data Explorer, Community > Action Campaigns	Real-time air quality data (PM2.5, PM10, NOx) for Qatar and the region, supporting health and environmental campaigns.
9	IEA (International Energy Agency)	Sectoral Contributions, Indicators and Statistics, Global Agreements	Tracks energy consumption, sectoral emissions, and energy efficiency metrics for mitigation strategies.
10	CDP Open Data (Cities & Corporates)	Partner Network, Call for Contributions, Policy and Reports	Disclosures from cities, corporates, and governments to support local action reporting and ESG integration.
11	Climate TRACE	Sectoral Contributions, Indicators and Statistics, Data Explorer	Satellite-based, facility-level emissions data for high-emission sectors like oil & gas, cement, and power.
12	World Bank API	Country Profiles, Regional Profiles, National Policies, Climate Finance	Provides development indicators, finance, population, GDP, and environmental policy context for holistic analysis.
13	OECD API	Policy and Reports, Climate Finance, Regional Frameworks	Tracks environmental instruments, subsidies, and climate finance flows to support policy and funding analysis.
14	GCF Project Explorer	Policy and Reports, Global Agreements, Community > Partner Network	Details on funded projects, disbursement status, and adaptation/mitigation focus for transparency.
15	UN OCHA HDX API	Regional Profiles, Community > Action Campaigns	Humanitarian and climate risk overlays for vulnerability and resilience mapping in crisis-prone areas.

16	OpenStreetMap / Overpass API	Country Profiles, Regional Profiles, Data Explorer	Geospatial mapping of facilities (e.g., energy infrastructure, natural features) for spatial analysis.
17	Google Earth Engine	Data Explorer, Indicators and Statistics, Solutions and Tools	Monitors land use changes, temperature maps, and NDVI for drought/vegetation indices, aiding adaptation planning.
18	Hugging Face API (NLP Models)	AI Search Tool, Glossary, FAQs	Enables semantic search, multilingual query understanding, and auto-summarization for user accessibility.
19	Wikidata SPARQL / API	Climate Change Agreements, Renowned Organizations, Glossary	Enriches background data by linking COPs to UN bodies and organizations for contextual clarity.
20	Carbon Disclosure Project (CDP) API	Climate Finance, ESG, Policy and Reports	Provides corporate and city-level ESG performance data, including carbon footprints and sustainability metrics, to support investor and policy decisions.
21	S&P Global ESG API	ESG, Climate Finance, Indicators and Statistics	Delivers ESG scores, risk assessments, and climate-related financial data for companies, aiding investment and risk analysis.
22	UN SDG Indicators API	Indicators and Statistics, Global Agreements, Policy and Reports	Tracks progress on SDG goals (e.g., SDG 13: Climate Action) with global and regional datasets for alignment with sustainability targets.
23	Climate Policy Radar API	National Policies, Policy and Reports, Global Agreements	Uses AI to analyze climate policies and commitments, providing insights into mitigation and adaptation strategies.
24	Global Carbon Project API	Indicators and Statistics, Sectoral Contributions	Offers global and national carbon budgets, emissions trends, and sectoral breakdowns for mitigation tracking.
25	MSCI Climate Solutions API	Climate Finance, ESG, Solutions and Tools	Provides climate risk metrics, portfolio carbon analysis, and green investment data for financial decision-making.
26	Task Force on Climate-related Financial Disclosures (TCFD) Data Hub	Climate Finance, ESG, Policy and Reports	Aggregates corporate disclosures on climate risks and opportunities, supporting financial transparency and risk management.
27	Adaptation Fund Project Database	Policy and Reports, Community > Partner Network, Adaptation	Tracks adaptation-focused projects, funding allocations, and outcomes for resilience-building initiatives.
28	ClimateWatch Adaptation Data	Adaptation, Indicators and Statistics, Country Profiles	Provides data on adaptation plans, vulnerability assessments, and resilience metrics for national and regional strategies.
29	Global Facility for Disaster Reduction and Recovery (GFDRR) API	Climate Risk, Regional Profiles, Solutions and Tools	Supplies risk profiles, disaster recovery data, and resilience tools for climate risk management.
30	UNEP-WCMC Online Reporting System API	Policy and Reports, Global Agreements, Data Explorer	Enables access to Multilateral Environmental Agreement (MEA) data, including response rates and questionnaire responses for climate-related reporting. Supports dynamic display of live data on the portal.
31	UNEP-WCMC Species+ API	Indicators and Statistics, Data Explorer, Solutions and Tools	Provides data on biodiversity and species impacted by climate change (e.g., CITES-listed species). Useful for integrating biodiversity metrics into adaptation and ecosystem resilience strategies.
32	UNEP IMEO Methane Alert and Response System (MARS)	Sectoral Contributions, Indicators and Statistics, Community > Action Campaigns	Delivers satellite-based methane emissions data, enabling monitoring and notifications for mitigation in high-emission sectors like oil and gas.
33	IPCC Interactive Atlas API	Climate Data, Indicators and Statistics, Solutions and Tools	Provides access to climate model projections and regional data from the IPCC Sixth Assessment Report (AR6). Supports visualization of temperature, precipitation, and extreme event scenarios for risk and adaptation planning.
34	UNIDO Open Data Platform API	Sectoral Contributions, Policy and Reports, Solutions and Tools	Offers data on sustainable industrial projects, energy efficiency, and cleaner production initiatives. Supports mitigation and SDG 9 (Industry, Innovation, and Infrastructure) tracking.
35	ESCWA Climate Change Data Portal API	Regional Profiles, National Policies, Climate Finance	Provides Arab-region-specific data on climate actions, circular carbon economy, and adaptation in water and energy sectors. Enhances regional policy and finance analysis.
36	UNSD SDGs API	Indicators and Statistics, Global Agreements, Policy and Reports	Tracks progress on SDG goals, particularly SDG 13 (Climate Action), SDG 7 (Affordable and Clean Energy), and SDG 11 (Sustainable Cities). Integrates global and regional sustainability metrics.
37	UNDP Data API	Country Profiles, Regional Profiles, Community > Partner Network	Supplies development and climate resilience data, including project outcomes and vulnerability indicators. Supports adaptation and community-focused initiatives.
38	UNdata API	Indicators and Statistics, Country Profiles, Regional Frameworks	Provides access to a wide range of UN datasets, including climate, energy, and environmental indicators, for cross-cutting analysis and dashboard integration.

Integration approach: For all these APIs, we should build a backend service or use ETL (extract-transform-load) jobs that retrieve the data, transform it to our schema, and store it in our database. Caching is vital – many APIs have rate limits or performance considerations, so we won't call them for every page load. Instead, schedule updates (with a cron job or triggered by a webhook if the provider offers one). We'll maintain attribution for data sources, and metadata like the last updated date for each dataset (perhaps shown in a tooltip or in the **Indicators** section). Error handling is also important – if an API is down or returns invalid data, the system should fallback gracefully (e.g., use the last known data and flag it). By integrating these sources, the portal ensures that its figures are credible and current, meeting the needs of data architects and policymakers who rely on accuracy.

2.2. Ingestion of National/Regional Data

In addition to external APIs, the portal will ingest data from national and regional entities that may not be available through those global APIs. For example, Qatar might have specific datasets (e.g. from MoECC) such as detailed energy usage, climate adaptation

project data, or environmental indices specific to the Gulf region. We need a flexible data ingestion pipeline to handle such data, which could come in formats like CSV, Excel, or JSON exports. This can involve:

- Setting up a process for content managers or data managers to upload datasets via an admin interface. For instance, if a new report on Qatar’s air quality is released with data tables, the admin can upload the CSV into the system. The backend should then parse and store it, making it available for visualization in the portal.
- Writing custom scrapers or connectors for regional open data portals. Some GCC countries have open data sites; if we plan to include regional profiles, we might fetch data from those (assuming permission). Alternatively, regional organizations (e.g., GCC Statistical Centre) might provide data – coordinate to obtain it and load it in our database.
- Normalizing and standardizing data formats during ingestion. Different sources might use different units or category names. A data architect will define a canonical schema (for example, ensure all emission data uses the same unit and gas categories). The ingestion process will include transformations to fit data into that schema, with documentation of assumptions.
- Handling geospatial data: If we ingest GIS data (e.g., shapefiles for climate zones or flood risk areas in Qatar), we should convert those to an open standard like GeoJSON and store or index them for use in mapping components. The system should be able to serve these via an API to front-end maps.
- Quality control: each ingested dataset should be reviewed. Perhaps maintain a log of data sources, with versioning (if a dataset is updated, we might want to keep the old version or at least note changes). Data architects might use scripts to validate ranges or catch anomalies.

The result is an internal data repository that combines global data (via API) and local data (via manual ingestion or custom sources). This repository underpins the Climate Data sections (profiles, explorer, indicators). We must also consider data **license and permissions** – ensure we have rights to republish data, especially for national sources. We will attribute data sources in context (like a footnote on a chart). Technically, using a relational database (like PostgreSQL) with PostGIS for spatial data, and/or a time-series DB for certain high-frequency data, could be part of the architecture. The ingestion scripts could be written in Python (pandas for CSV, etc.) or Node, depending on team expertise, and possibly run as scheduled jobs or on-demand when an admin triggers an update.

2.3. Metadata/Tagging Taxonomy for Search and Indexing

To enable powerful search and AI functionalities, a well-designed metadata and tagging system is essential. We will implement a **controlled vocabulary** (taxonomy) for the portal’s content. This includes:

- **Topic Tags:** Define a set of core topics (e.g., Mitigation, Adaptation, Energy, Water, Health, Policy, Finance, Education, etc.). Every content piece (be it a page, news article, or dataset) can be tagged with one or multiple topics if relevant. For instance, a news article about a solar project would get tags like “Energy” and “Mitigation”.
- **Geographical Tags:** Use country and region tags. We’ll have a tag for “Qatar” that is applied to content specifically about Qatar, and tags for other countries or “Global” as needed. This way, search queries constrained by location can be handled (the AI could pick up context like “in Qatar” to filter results to content with the Qatar tag).
- **Content Type Tags:** Label content by type (e.g., “Policy Document”, “Data Set”, “News”, “FAQ”). This aids the search engine in diversifying results (or if we later allow filtering search by type). The taxonomy manager in the CMS should support hierarchical tags too – e.g., under “Policy” we might have sub-tags like “Law”, “Strategy”, “International Commitment”.
- **Semantic Enrichment:** Beyond manual tags, we’ll leverage AI to auto-tag and enrich metadata. Using AI tagging techniques, the system can scan text and assign additional tags or keywords that content managers might not have explicitly added. For example, an article might implicitly talk about “greenhouse gases” and “transportation” – an AI NLP service can detect those concepts and tag the content with “GHG” and “Transport” even if not manually tagged. These AI-generated tags (subject to review for accuracy) can significantly improve search recall.
- **Standards and Schema:** We will adhere to common metadata standards where possible. For documents and data, Dublin Core fields (title, creator, date, subject, description) can be stored. For geographic tagging, ISO country codes will be used. If we expose content via an API or for interoperability, using schema.org vocabularies for things like Article, Dataset, etc., will help other services parse our metadata. Internally, the taxonomy might be managed in the CMS taxonomy module or a dedicated ontology service.
- **Knowledge Graph/Ontology:** In the architecture, consider maintaining a lightweight knowledge graph linking entities (this is forward-looking). For instance, we treat “Paris Agreement” as an entity and link it to content (like a node connected to the Paris Agreement page, Qatar’s NDC, etc.). This could greatly enhance the AI search by

providing context relationships. The taxonomy/ontology management system ensures consistent tagging and can ensure search precision (e.g., if a user searches a colloquial term, we know its equivalent in our tags).

All content creation and editing workflows should prompt the user (content manager) to apply tags from the taxonomy. This ensures nothing falls through un-tagged. For indexing, our search engine (be it Elasticsearch or a vector store for AI) will ingest both the full text and these tags/metadata. We should configure search weighting such that matches in tags/metadata might boost relevance (because if something is explicitly tagged, it's very relevant to that topic). The AI search pipeline will use metadata too – for example, if the user query mentions a category, we might filter the corpus by that tag before vector search, to reduce noise.

In summary, a governed tagging system combined with AI-driven metadata enrichment will make the portal's vast content easily navigable. This hybrid approach (human-defined taxonomy + AI auto-tagging) caters to the needs of content managers (who need consistency) and developers (who need data to be queryable and connected). It's the backbone for the "smart search" capability of the portal.

2.4. Infographic/Media Handling Capabilities

The portal will host a variety of media (images, infographics, videos) to enrich content. We need a robust way to store, retrieve, and display these media assets:

- **Image Library:** All images (photos, diagrams, charts) uploaded should be managed in a media library. The system should auto-generate different sizes for responsive use (thumbnails, medium for inline, full-size for lightbox) and optimize for web (compress without losing critical quality). We should also store metadata for images: captions, alt text (for accessibility), source/credit, and tags. Tagging images (e.g., "flood map", "workshop photo") can allow the AI to also retrieve or describe images if needed, and it helps content managers find them for reuse. If possible, integrate an image CDN or caching layer for faster delivery, especially since images might be high-resolution infographics.
- **Infographics and Custom Graphics:** Often, climate portals include infographic PDFs or interactive visualizations. For static infographics (say a PDF or image summarizing climate impacts), we can either embed them as images or provide as downloads. Ensure the PDF viewer or image zoom feature is user-friendly (some infographics are large and text-heavy, needing zoom/pan). If infographics are interactive (maybe built with Flourish, Tableau, or similar), ensure the embedding works (which might be via <iframe> or JS include). The CMS should allow embedding external media safely (with whitelisted domains, etc.).
- **Videos:** If videos are part of content (like recorded talks, or explanatory animations), decide whether to self-host or use a platform like YouTube/Vimeo for streaming. Using an external platform's embed is easier for bandwidth (e.g., embed YouTube players for any climate awareness videos). If privacy or control is an issue, self-host short videos and use HTML5 video. But self-hosting requires encoding to multiple resolutions and could strain server load. Likely using YouTube (maybe a dedicated channel for the portal) is a good approach. The portal front-end should have a lightbox or player component ready for embedded videos.
- **Interactive Media/Charts:** For interactive charts not covered by the Data Explorer – e.g., a bespoke visualization on a page – we need to integrate JS libraries (like D3 or Chart.js) or embed from tools (like an iframe from Datawrapper or Tableau Public). The site should allow such scripts in a secure way. Possibly, content managers might need developer help to create custom infographics, but having a system for it is useful. Perhaps the "Solutions and Tools" or "Climate Data" pages will use interactive components directly coded by developers.
- **Media Retrieval and Indexing:** We plan to make media also searchable. For instance, the AI might get a question "show me a chart of Qatar's emissions trend". If we have an infographic or chart image for that, ideally the search could surface it. Implementing this might involve tagging the chart image with "Qatar emissions trend" and the search engine including media metadata in its index. Alternatively, store transcripts or descriptions of multimedia (for example, provide text transcripts for videos for accessibility and search indexing). If OCR is feasible, we could even index text within PDFs or infographics (some infographics might have a lot of text). This ensures nothing is "hidden" from the search just because it's in an image.
- **Access Control:** Not specifically requested, but if any media or docs are meant for only certain users (maybe not, since it's a public portal), then a mechanism to restrict content might be needed (like requiring login to see certain images). We likely won't need this at launch unless some data is sensitive.

Summing up, the portal's media handling will be akin to a digital asset management lite – ensuring images and videos are easy to upload, correctly formatted for web, described for accessibility, and categorized for retrieval. Developers will ensure efficient delivery and compatibility (cross-browser, mobile-friendly media display), and content managers will ensure that each media enhances the user experience (with proper captions and attributions).

2.5. User Content Submission Workflow with Moderation

As outlined in the **Community > Call for Contributions**, we need a clear workflow for user-submitted content. Contributed content might include articles, data, event suggestions, or feedback:

- **Submission Interface:** Provide forms for the types of submissions we allow. For example, a form for submitting an article or case study would have fields like title, author name, email, content (or file upload), and perhaps category of content. We might have separate forms for different contribution types (one for general articles, one for data sets, one for event suggestions) to make processing easier.
- **Storage as Draft:** When a user submits, the data should go into a pending state in our system. If using a CMS, we could have a specific section for “User Submissions” that is not publicly visible until reviewed. The submission can be automatically formatted into our content structure (e.g., create a draft news article from the input).
- **Moderation Dashboard:** Admins (content managers or editors) need an interface to review submissions. This should list new submissions, with quick actions: view, approve/publish, request changes, or reject. If possible, integrate an email notification system: e.g., when a submission comes in, email moderators; if an editor marks “request changes”, maybe email the contributor to revise (this may require user accounts or at least capturing their email in the form).
- **Editorial Review:** Likely, editors will clean up or fact-check submissions. They might edit the text for clarity or correctness before publishing. The system should allow them to edit the content just like any other page or article, then flip a status to publish it live. The contributor should be credited (so have a field for author name that we can display).
- **Preventing Abuse:** Implement CAPTCHA on submission forms to prevent spam bots. Possibly require users to create a simple account or verify email for larger submissions, which adds friction but increases quality. At least, a confirmation email to the submitter (“Thank you, we received your contribution”) is good practice and can verify their email.
- **Content Placement:** Determine where approved contributions go. Perhaps community-submitted articles appear in a “Community Contributions” section under Resources or Community. Or if very relevant, they could be merged into other sections (for instance, a really high-quality research piece could go into Publications). We might have a tag or marker for community content so it’s clear what originated from outside the core team.
- **Licensing and Rights:** We should have contributors agree to terms (e.g., they have rights to whatever they submit and allow us to publish it). A checkbox on submission can cover this (“I agree that this content may be edited and published on the portal”). This legal aspect protects the project when publishing third-party content.

Through this moderated workflow, the portal taps into external knowledge while maintaining quality control. From a technical architecture perspective, if using a CMS like Strapi or Drupal, we’d leverage built-in roles/permissions and content states (draft, needs review, published) to facilitate this. The process needs to be streamlined so that moderation doesn’t become too time-consuming – possibly by providing editors with tools like side-by-side view of submission vs. how it will look published, etc. This addresses the requirement for community input while ensuring the portal remains a reliable source (no unchecked info gets on site).

2.6. AI-Powered Indexing and Smart Search

Building on the earlier points about the AI search tool and metadata, here we detail how AI enhances our indexing and search beyond traditional methods:

- **Semantic Indexing:** In addition to keyword indexing (which we’ll have via a search engine like Elasticsearch), we employ NLP techniques to index content by meaning. This involves generating embeddings for each content piece using a language model. We can use an open model (like SBERT or similar) or a service to compute these embeddings. All textual content – including titles, body text, summaries, tags – will feed into this. The resulting vector index lets us find related content even if exact keywords differ (e.g., a search for “rainfall trends” can match a content piece describing “precipitation changes” because semantically they’re close).
- **Enhanced Search Algorithms:** We might implement a hybrid search: first, use keyword search to narrow down possibly relevant docs (especially if a user uses some exact terms or filters), then use semantic ranking to sort or a secondary vector search to catch others. The AI search pipeline we design (maybe following a RAG pattern) should smartly decide how to retrieve context. For direct user consumption, we likely have the chatbot style answer. For the normal site search bar (if any separate from AI tool), we can also incorporate semantic suggestions (“Did you mean...”) or at least synonyms from our taxonomy.

- **Continuous Learning:** We can collect search query analytics and user behavior (clicks, whether they found what they need). Over time, this could feed into refining the search. Possibly incorporate an AI re-ranking model that learns from click-through (this is advanced and might be future consideration). At least manual tuning is possible: if we see the AI giving irrelevant answers for certain queries, we adjust content or add an FAQ to address it.
- **Multilingual Smart Search:** Ensure the search index and AI cover both languages. This might involve maintaining separate indices for English and Arabic content. If a user queries in Arabic, we use the Arabic index and an Arabic embedding model (or a multilingual model capable of cross-language retrieval). For example, we could use a multilingual model that places both language embeddings in the same vector space, enabling cross-lingual retrieval (so an Arabic question could retrieve an English document if no Arabic available). This would greatly benefit cases where content is only in one language. We should also use language detection on queries to route them appropriately.
- **Contextual and Fuzzy Search:** Implement features like spelling correction or tolerance (if someone types “Qatar emission”), the search should correct to “Qatar”. Also, allow the AI to handle natural language questions that might combine multiple facets (e.g., “What are Qatar’s 2030 emission targets and are we on track?”). Answering that might involve pulling from Sustainability Targets and Indicators. This is where retrieval augmented generation shines, as the AI can piece together info from multiple sources if we provide those passages.
- **Performance and Scaling:** Indexing all this data (including possibly large PDFs and data) means we need efficient indexing pipelines. We might schedule indexing jobs when content is updated. If using a vector DB, we need to manage its size and memory. Maybe limit embedding to textual content (not raw data tables unless needed). Using cloud-based managed search services could help scalability. We should also consider response time: semantic searches and LLM responses are heavier than simple DB queries. Caching common queries or pre-generating some Q&A pairs (like from FAQs) could speed up response for frequent questions.

Overall, AI-powered indexing and search is what transforms the portal from a static library into an interactive knowledge system. It will significantly improve user experience by quickly connecting them to the information they seek, even if they don’t know where to find it. Our technical implementation must ensure accuracy (grounding answers in real content) to maintain trust – hence the emphasis on retrieval with context to avoid hallucinations.

2.7. English–Arabic Multilingual Support

The portal will fully support both English and Arabic, reflecting Qatar’s bilingual context. This has several implications:

- **Content Storage:** Each piece of content (page, article, etc.) needs translations. In a CMS, we might have a multilingual setup where each content item can have an English and Arabic version under the hood. Alternatively, we create parallel content entries linked together (one for EN, one for AR). The system should enforce that critical content is translated or at least mark what isn’t. For some dynamic data (like numbers, charts), the data is language-agnostic, but labels, captions, and descriptions need translation.
- **Localization of UI:** All user interface elements (menus, buttons, prompts, form labels) should appear in the user’s chosen language. We will maintain resource files for UI strings, enabling easy updates and ensuring consistency. This includes date formats (perhaps use Arabic numerals and Arabic month names in Arabic mode) and other locale-specific formats.
- **RTL Layout:** The Arabic side will use a right-to-left layout. This means our frontend CSS needs to support mirroring of the layout. Many modern frameworks allow an RTL stylesheet or a toggle. We must ensure everything looks proper in RTL – for instance, menus and sidebars should flip horizontally. As noted, some graphical elements need not flip (like icons of arrows might need to point appropriate direction). We will thoroughly test the Arabic interface for such nuances.
- **Switching Language:** Provide an easy toggle (likely a language switch link or button) that persists user choice (possibly via cookie). Users should be able to switch at any page and see the equivalent page in the other language if it exists. If a page isn’t translated yet, we should decide: hide it in Arabic menu or show English content as fallback? Probably better to hide or have a placeholder “content coming soon in Arabic” for professionalism, but ideally have all content translated before launch.
- **Search and AI in Arabic:** As discussed, the search needs bilingual capabilities. We’ll implement a separate Arabic search index and use Arabic NLP for the AI. This involves using Arabic morphological analysis for keywords (Arabic has complex forms, so using Elastic with an Arabic analyzer for keyword search, and an Arabic embedding model for semantic search). Ensuring the AI doesn’t just translate the question and answer in English is important; we want native Arabic answers. If using GPT-like models, we’ll prompt them to answer in Arabic when the query is Arabic.

- **Data Visualization Localization:** Numerals in charts, axes labels, etc., might need to appear in Arabic (e.g., Arabic-script digits and labels). We should use libraries that support locale or dynamically change the text. If we embed Tableau or similar, it might not localize text unless multiple versions are provided. For custom charts, we can label in Arabic easily by providing translated strings. Map labels (like country names) could also be shown in Arabic if we have those; maybe use bilingual maps or just English maps with possibly an Arabic legend. This is a detail depending on how our mapping is done (we could use localized map tile services).
- **Testing Content Flow:** Some languages when translated can expand text length. Arabic might be longer or shorter for different phrases. We should test key pages to ensure layout still looks good with Arabic text (no overflow issues). Also check that our fonts support Arabic script nicely (use a web font that is legible).

Multilingual support adds complexity, but it's crucial for regional impact. By addressing UI, data, and search localization, we ensure the portal is equally effective for Arabic-speaking stakeholders. Developers must implement toggling and mirrored layouts, while content managers must maintain two versions of content. Also, any user-generated content (like contributions or comments) should capture what language they are in and display accordingly. In sum, everything from the **architecture** (content models) to **frontend design** to **search algorithms** will account for English–Arabic duality, delivering a seamless experience in both languages.

3. Technical Architecture Recommendations

3.1. Scalable CMS & Knowledge Base Backend

We recommend using a **headless CMS** as the backbone for managing content, combined with custom services for data handling. A headless CMS (such as Strapi, Contentful, or Drupal with JSON API) will allow content managers to create and update pages, articles, glossary terms, etc., through a friendly interface, while exposing content via APIs to our front-end. This decoupled architecture ensures scalability – the content repository can grow (even serve multiple front-ends or country sites in the future) and developers can independently scale or modify the presentation layer. The CMS should support our complex content model: multi-level pages (for sitemap structure), multi-language fields, and a robust taxonomy system. For example, **Drupal** could be a candidate given its strong multi-language and taxonomy features (and it can be run headless), or **Strapi** for a more modern JS stack with plugin support for i18n. We will also build custom backend services (using Node.js with Express or Python with Django/Flask) for the dynamic data aspects – such as the data explorer API endpoints, search indexing tasks, and integration with external APIs. Essentially, think of two backends: one is the CMS for content, the other is a data API layer for climate datasets and search. To make the knowledge base aspect effective, we also consider incorporating a **graph database** or knowledge graph module to model relationships between content (for semantic queries). However, that can be an augmentation to the CMS rather than a separate component initially. Key is that the chosen backend stack is **scalable**: it should handle increasing load (both in content volume and user traffic). Using a cloud-based database (SQL for content, NoSQL or graph for specialized needs) with proper indexing, and containerizing the services for deployment (Docker/Kubernetes), will allow us to scale horizontally if needed (e.g., spin up more instances of the API service during high load). Also, implementing caching at the API layer (using Redis or CMS built-in caching) will improve performance for frequently requested content. This architecture can serve the entire MENA region by replicating the model for different countries (multi-tenant CMS where each country is a site, or separate instances – headless makes either possible). The bottom line: a **modular, service-oriented backend** where the CMS handles structured content management and a custom data service handles dynamic climate data and heavy logic (search, calculations) will provide the needed scalability and flexibility for a growing knowledge hub.

3.2. Tableau Dashboard Embedding

For advanced visualizations and dashboards, especially those that stakeholders (like government analysts) already use, we plan to embed **Tableau** dashboards into the portal. If the climate portal team or partners have prepared dashboards (e.g., a dashboard of climate indicators or scenarios), embedding them avoids reinventing complex visuals and allows live updates from the Tableau side. To do this, we will use Tableau's embedding API/iframe. Tableau Server or Tableau Public can provide a URL for each dashboard that we integrate in an `<iframe>` element on a page. Key recommendations for this:

- Ensure that the embedding is responsive – Tableau iframes can be made to resize or we might set a max width. We might use the Tableau JavaScript API to control parameters (like hiding toolbar, setting language if needed, etc.).
- If the dashboards are on a secured server (not public), we'll need to handle authentication. Options include using trusted tickets or OAuth if the user needs to login. But since this is a public portal, we might either host dashboards on Tableau Public (which doesn't require login but data is public) or on a Tableau Server configured to allow guest access to those specific views.

- Align the look and feel: we should use Tableau’s capability to have a transparent background or matching color scheme so it blends with our site design. At minimum, disable default Tableau padding and use custom CSS if necessary to ensure it looks integrated.
- Provide alternatives for users who cannot see the embedded content (some users behind strict networks or with JS disabled). At least a direct link “View full dashboard” should be provided to open it in a new tab.
- Example use: an interactive map of Qatar with climate vulnerability indices by region done in Tableau can be embedded in the Country Profile or Solutions section. Users can hover and filter within that iframe as if they were on Tableau. We need to test such interactions on various devices to ensure usability (Tableau charts might be less usable on small screens; if so, consider designing simpler mobile views or providing a summary).

Additionally, we might embed other external tools similarly (ArcGIS dashboards, Flourish charts, etc.); the approach is similar. Embedding saves development time for complex interactivity but we must monitor performance – if an embedded dashboard is heavy or slow to load, it might affect our page. We can use lazy loading (load the iframe only when the user scrolls to that section or clicks “view dashboard”) to mitigate initial load time. By embracing these embed techniques, the portal can showcase rich, data-heavy visuals that are maintained by data analysts in Tableau, combining the best of CMS content and BI tools.

3.3. GPT-Style Search Integration

To build the intelligent search experience, our architecture will incorporate a **retrieval-augmented generation (RAG)** pipeline using GPT-like models. The recommendation is to set this up as a microservice or part of the backend dedicated to search/query processing. The components and flow are:

1. **Indexing:** As detailed, use a vector database (such as Pinecone, Weaviate, or even Elasticsearch with vector capabilities) to store embeddings of all content. This search service will interface with that DB.
2. **Query handling:** When a user asks a question via the AI search tool, the query goes to the search service. We first perform a semantic search for relevant documents/passages. This yields the top K pieces of text likely to contain the answer.
3. **Generation:** Feed those retrieved contexts into a GPT-like model (could be OpenAI’s GPT-4 via API, Azure OpenAI, or a locally hosted model depending on policy and cost considerations) with a prompt that includes the user’s question and the context, asking the model to formulate an answer using the provided information. This step produces a natural language answer.
4. **Post-processing:** The service can then format the answer (including citations to sources if we want transparency) and return it to the front-end. Also, if the model fails to find an answer or expresses uncertainty, we can have fallback options (maybe show related content links).

To integrate this seamlessly: The front-end (likely using a React component for chat Q&A) communicates with this backend via an API endpoint (e.g., /api/search/ask). We should design the API contract for this, including passing the language context (so it knows to use the Arabic model vs English model).

Performance and scalability are concerns: calling a large model for every query can be slow and costly. We can mitigate this by:

- Using caching: If the same question was asked recently, cache the answer for quick retrieval.
- Fine-tuning smaller models on our corpus for a more efficient on-premise solution, if feasible. Or use a smaller open-source model (like LLaMa 2 fine-tuned for Q&A) hosted on our server for faster responses, albeit perhaps with lower quality than GPT-4. A hybrid approach might use GPT-4 initially and see if a distilled model could serve later.
- Ensuring the retrieval step narrows down context effectively to reduce the token load on the GPT model (there are limits like prompt length).

The integration also involves the **UI/UX**: We have to incorporate this like a chat or an expanded search result on the front-end. It should handle multi-turn if we allow follow-up questions (that requires keeping context of the conversation – which means the backend would need to maintain state or the front-end sends conversation history). Even if not full multi-turn chat, allow the user to refine query and get a new answer.

A potential architecture enhancement is to incorporate Microsoft’s or another search vendor’s cognitive search which has built-in integration with OpenAI (for example, Azure Cognitive Search can index content and then an Azure OpenAI can be used to answer, as referenced in some MS documentation). This could reduce custom code if we leverage such a service, but also ties us to a vendor. Otherwise, implementing with open-source libraries like **Haystack** (which is designed for RAG Q&A) can speed up development.

In summary, the GPT-style search integration will be a distinct part of the system focusing on **NLU (Natural Language Understanding)**. It must interface well with our content (via indexes) and be optimized for our knowledge domain. The architecture

should treat it as an asynchronous service possibly, to not block the main site operations. By doing this, we deliver cutting-edge search capability that aligns with modern user expectations of interacting with knowledge bases in a conversational way.

3.4. API/Webhook Integration with External Platforms

The portal will not exist in isolation – it will both consume external updates and potentially send out updates. Architecturally, we should set up:

- **Incoming Webhooks/Subscriptions:** If available, subscribe to data updates. For example, if WRI's API could send a webhook when they update a dataset or if a news source provides a webhook for new articles, our system should catch that and trigger a data import or content update. A concrete case: maybe MoECC has an internal system that can ping the portal when a new policy is approved, so the portal can automatically create a placeholder for it. We can create endpoint listeners in our backend to handle such hooks and tie them to ingestion scripts. This makes data integration more real-time without waiting for scheduled jobs.
- **Outgoing Webhooks/Notifications:** When our portal content updates, we might want to notify external systems or stakeholders. For instance, if the portal adds a new event, perhaps push it to a public Google Calendar or send a tweet via an integration. Or if we have partner sites (like the QF website) that want to display our latest news, we could provide a webhook or RSS feed for them. While not an initial requirement, structuring our backend to easily output feeds (RSS/Atom/JSON feeds for news, events, datasets) is recommended to facilitate easy sharing. It aligns with open data principles – letting others consume what we have.
- **Internal Hooks:** Within our system, use an event-driven approach for decoupling components. For example, when a content piece is published, emit an event that triggers re-indexing for search. Or when a dataset is updated, fire an event to update relevant indicators or send an email to interested users. This can be achieved with a message queue (like RabbitMQ or even simple pub-sub pattern in the application). This internal webhook-like system ensures the various modules (CMS, data, search) remain loosely coupled but in sync.

In practical terms, suppose UNFCCC releases a new GHG inventory submission. Maybe they don't have a webhook, but we schedule a check. However, for something like news, perhaps partnering with a service that does have push notifications (like subscribing to a news agency's feed). The architecture should thus include a **scheduler and webhook processor** service. We might run a cron job (via something like Celery for Python or node-cron for Node) for periodic tasks, and a small service or background worker to handle incoming webhooks instantly.

Security is key for incoming webhooks – we validate signatures or tokens to ensure it's a trusted source calling our endpoint. Also, we queue heavy tasks to not tie up webserver threads (for instance, an incoming webhook kicks off a job to fetch data and update DB asynchronously).

By integrating via hooks, we keep the portal's data **fresh** and also make it a source of truth that others can tap into. The regional scale suggests that down the line, multiple national portals could interconnect – e.g., a GCC-wide dashboard could consume our portal's API. Thus, exposing a well-designed REST/GraphQL API for our content is a forward-thinking recommendation. Even if not used by many initially, an open API aligns with transparency (others could query our indicators or read our published content via API instead of scraping). So in the architecture, define clear API endpoints (with documentation) for key data (like `/api/data/indicator?country=QAT&name=GHG` emissions returns a JSON of that time series, etc.).

3.5. Responsive, Modular Frontend (Scalable Across MENA)

The frontend of the portal should be built with a modern framework that supports component-based development, routing, and fast rendering. We recommend using a framework like **React** (possibly with Next.js for server-side rendering and SEO benefits) or **Angular/Vue** if the team has expertise. React with Next.js might be ideal: it allows pre-rendering of static content pages (which helps with SEO and initial load), and can also do client-side hydration for the dynamic parts (like interactive charts or the chatbot). A modular design means each section of the site (About, Profiles, Data Explorer, etc.) is implemented as a set of components that can be developed and tested independently. This also helps if we expand the portal to other countries – we could reuse components and just feed them different data via the CMS or API.

- **Responsive Design:** Mobile-first approach should be taken given many users might browse on smartphones. The layout should adapt for various screen sizes: for instance, the main menu might collapse into a burger menu on small screens; multi-column data or charts might stack or use sliders. Testing each page type on mobile is crucial (e.g., data tables might need a horizontal scroll on mobile, charts might need simpler presentation). Use CSS frameworks or utilities (like Bootstrap 5 or Tailwind CSS) to expedite responsive development, but customized to the portal's branding.
- **RTL Support on Frontend:** As noted, ensure the CSS and component logic can flip for Arabic. This might involve adding `dir="rtl"` on the `<html>` when Arabic is active and having style rules that auto-adjust (some frameworks have

RTL versions). We might maintain two sets of style files if needed but ideally a well-planned single stylesheet can handle both using logical properties (CSS that uses start/end instead of left/right).

- **Theming and Scalability:** Since the portal may expand regionally, consider a theming mechanism where colors, logos, etc., could be adjusted if, say, another country uses the same codebase but different branding. Using CSS variables or theme context can allow quick re-skinning. Also, structure the content in the CMS such that each country's content is separable (if needed).
- **Performance:** Use lazy loading for heavy components (load them only when needed). For example, the AI chatbot component might load its heavy scripts only when the user opens the chat interface. Similarly, load images with loading="lazy" and perhaps use progressive loading for maps (show a simple image or spinner while a heavy map loads). Bundle optimization and using a CDN for static assets will improve load times across the region. Also, because internet speeds and devices vary, keep the site lightweight where possible (minimize massive libraries, optimize images).
- **Accessibility:** Follow WCAG guidelines to ensure the portal is accessible (alt tags on images, keyboard navigability, contrast ratio for text, etc.). This also inherently improves the semantic structure of the site, which can help search engines index it properly (important even if we have AI search, because people might still find pages via Google).
- **Testing and Browser Support:** Ensure it works on commonly used browsers in the region (likely Chrome, Safari, mobile browsers, maybe older IE is not needed given it's 2025, but ensure Edge/IE11 if some government users still use older browsers). Use polyfills or fallbacks if needed for features (though with modern frameworks and dropping IE, we can mostly use modern JS).

By building a modular responsive front-end, the portal will be easily maintainable and extendable. If new features are required, we can develop new components and plug them in without rewriting whole pages. If new countries join the initiative, we can spin up a new instance of the front-end pointing to a different content source or use a single multi-tenant front-end that loads data based on selected country. Essentially, a **component-based architecture** for front-end aligns with scaling the content across the MENA region with minimal duplication of effort.

3.6. Compliance with Open Data Standards (CSV, JSON, XML, GeoJSON)

Both in data intake and output, the portal should comply with open data standards to facilitate interoperability. This means:

- Any data downloads we offer to users (in the Data Explorer or elsewhere) should be in common formats like CSV for tabular data or GeoJSON for map data. For example, if a user wants to download a chart's data, they get a well-formatted CSV with a header row describing columns (in English, or both languages if necessary). We should ensure encoding is UTF-8 on these files to handle Arabic text if included.
- If we provide an API for data (which would likely return JSON), it should follow typical REST conventions and possibly standards like the OGC (Open Geospatial Consortium) standards for any spatial data. GeoJSON is mentioned, which is great for vector data (points, polygons etc.). If we ever serve time-series geospatial data, we might consider formats like NetCDF, but that's probably beyond portal scope (NetCDF might be used internally via CCKP but we'd convert to CSV/JSON for users).
- Provide metadata along with data: For instance, include units and definitions in an "about" or "metadata" section for each dataset. This could be a JSON metadata structure following something like Data Catalog Vocabulary (DCAT) or simply a descriptive JSON block attached to downloads. If our portal is to be machine-friendly, consider publishing a metadata catalog (maybe an XML or JSON file listing all available data sets with schema info).
- Use ISO date formats in data (e.g., YYYY-MM-DD) to avoid confusion, and coordinate systems like WGS84 for spatial data. Essentially adopt widely used standards so that anyone using our data can easily integrate it into their tools (Excel, GIS software, etc.).
- Possibly adopt open licenses for data (like Creative Commons) as part of compliance with open-data best practices – this isn't a format but a policy to allow reuse.

From the developer side, ensure that when generating these outputs, we format them correctly. For example, generating CSV with proper escaping, or using libraries to output Excel if needed (some users prefer .xlsx – we might consider offering that as well, but CSV is more universal). For JSON, follow a consistent structure (maybe GeoJSON for spatial, and a simple array of objects for others). If we do anything with XML, maybe if we interface with older systems or publish an RSS feed (RSS is XML) for news/events, that should be well-formed and follow RSS 2.0 spec.

By adhering to these standards, the knowledge portal not only displays data but also becomes a data source for others. It aligns with open data principles and ensures that as the project grows (or as researchers use the portal's info), the data can be exchanged without format hurdles. This future-proofs the platform as a reputable hub for climate data in standardized formats.

3.7. Real-Time Data Display for Climate Indicators

In some cases, we want to display data that updates in real or near-real time. Examples might include current air quality index, live temperature readings, or a live counter of campaign participation. For such scenarios, the architecture should support pushing updates to the front-end without full page reloads:

- We can use WebSockets or Server-Sent Events (SSE) to push data from server to client. For instance, a WebSocket connection could broadcast updates of a certain indicator to all clients viewing a page. If we had a widget showing “Atmospheric CO₂ (ppm)” that updates daily, a scheduled job could fetch the new value and then the server pushes it via WebSocket to all open clients.
- On the front-end, we'll have a small client that subscribes to these updates and updates the DOM accordingly (for example, update the text or chart with the new value). This avoids the user needing to refresh to see changes. It's particularly useful if we had a dashboard screen somewhere that is meant to be displayed constantly (maybe an info kiosk) – it would always show current info.
- For not truly real-time but frequently updated data (like hourly or daily), we might not need persistent sockets – a periodic AJAX fetch could suffice. But since we aim to be modern, implementing a WebSocket for any live streams is wise. There might be future use-cases (like integrating live sensor data from an IoT network).
- If we plan any live interactive events or chats, WebSocket is needed, but that's beyond current scope.

An example: Suppose at COP, the portal shows a live feed of emissions pledged as countries announce them – such a thing could be done with live updates. Or a simpler example: an “Event is now live – watch here” banner that appears exactly when an event's start time passes.

The backend to support this might be a Node.js service with Socket.io (commonly used for realtime) or using a message broker + websocket server setup. For scaling, one might use a cloud service like AWS AppSync or Azure SignalR, but a self-hosted Socket.io or similar will work for moderate loads.

We should also ensure that the real-time component degrades gracefully: if WebSocket is not supported or fails, the client can fallback to polling every X seconds.

Real-time display will keep the portal feeling dynamic and “alive.” It's a feature that can set it apart by highlighting current data. We just need to choose carefully which indicators truly warrant it. Many climate indicators don't change second-to-second (like emissions data is annual). However, things like current weather/climate extremes, or a live counter for a campaign (“trees planted: 12345 and counting...”) would utilize it. We should build the capability so it's there when needed.